

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA SISTEME DISTRIBUITE ÎN INTERNET

LUCRARE DE DISERTAȚIE

**O interfață intuitivă pentru gestiunea fișierelor în
medii cloud cu un model specific de redundanță**

Conducător științific

Conf. Dr. Boian Rareș

Absolvent

Holcan Cosmin

2025

ABSTRACT

The purpose of this thesis is to explore how cloud solutions can simplify the process of file management for less experienced users through an intuitive web application. In the first part of the thesis, we examine key concepts surrounding cloud computing, including its architecture, models, and current storage solutions with a focus on file versioning and redundancy. We also discuss the principles of intuitive web application design, especially within cloud-based environments. The second part presents the proposed solution: a web application that simplifies file management while allowing users to choose specific redundancy zones, a feature not natively available through major providers like Azure or AWS. The application is designed to address two main objectives: creating a cloud-based file management tool that is accessible to non-technical users, and enabling the customization of redundancy zones to enhance data control, using a custom redundancy procedure.

Cuprins

1	Introducere	1
1.1	Obiective	1
1.2	Structura lucrării	2
2	Fundamente teoretice	3
2.1	Tehnologii cloud	3
2.1.1	Scurtă prezentare	3
2.1.2	Arhitectura cloud computing	4
2.1.3	Modele de implementare în cloud computing	6
2.2	Aplicațiile web	7
2.2.1	Structură	7
2.2.2	Aplicațiile web în mediile cloud	9
2.3	Practici în crearea de interfețe intuitive	10
3	Redundanța în sistemele distribuite	13
3.1	Principiile redundanței	13
3.2	Tehnici cheie de redundanță	14
3.2.1	Replicarea datelor	14
3.2.2	Redundanța geografică	15
3.2.3	Mirroring-ul datelor	15
3.3	Disponibilitate ridicată	17
3.4	Soluții existente	19
3.4.1	Redundanța în Azure	19
3.4.2	Redundanța în AWS	21
4	Descrierea aplicației	22
4.1	Interfața grafică și mod de utilizare	23
4.2	Arhitectura	27
4.3	Implementare	30
4.4	Testare	37
5	Concluzii	39
	Bibliografie	40

1 Introducere

Problema pe care o voi dezbate pe parcursul acestei lucrări este dată de utilizarea, or mai exact simplificarea acesteia, de către soluții cloud în gestionarea fișierelor de către utilizatori mai puțin experimentați. Tehnologia cloud a devenit în ultimii ani larg răspândită, atât de către companii, cât și de populația obișnuită, dar fără a conștientiza neapărat acest lucru, prin diferite aplicații precum Google Drive, Dropbox ori platformele sociale sau de streaming. Totuși, când vine vorba de gestionarea fișierelor, utilizatorii nu au un contact atât de mare de cum ar putea datele lor fi salvate, iar lucrarea mea se va concentra asupra acestui aspect, de cum ar putea o interfață intuitivă să ajute majoritatea populației într-un control mai mare asupra fișierelor pe care le salvează în medii cloud. În cadrul soluției propuse vor fi dezbătute de asemenea și alte teme mai specifice, cum ar fi crearea unui sistem de redundanță pornind de la soluțiile existente, menit să ofere utilizatorilor un grad mai mare de libertate în opțiunile luate cu privire la salvarea fișierelor.

Inspirația din spatele acestei teze constă din experiența personală în utilizarea serviciilor cloud direct de la marii furnizori de astfel de servicii. Astfel, am putut observa cum doar pentru o simplă stocare de fișiere sunt necesare destul de multe cunoștințe specifice pentru mediile cloud, în special pentru fiecare furnizor în parte și mi-am pus problema cum o abordare mai simplă pentru utilizatorii obișnuiți ar putea schimba acest proces. De la această problemă simplă și de la observarea altor lucruri care ar putea fi schimbate pentru o mai mare flexibilitate, cum ar fi selecția de zone de redundanță în mod specific de către utilizatori, am ajuns să dezvolt această aplicație menită să ofere o abordare mai practică în salvarea de fișiere în cloud.

1.1 Obiective

- Descrierea tehnologiei cloud, incluzând concepte cheie, arhitectură și soluții oferite în prezent

- Prezentarea stocării fișierelor în mediile cloud, opțiuni de stocare, versionare și în special de redundanță
- Ilustrarea principalelor concepte despre aplicații web, în special în mediile cloud, și a aspectelor de avut în vedere în crearea unei interfețe cât mai intuitive
- Dezvoltarea unei aplicații web cu ajutorul tehnologiilor cloud venite să ajute utilizatorii neantrenați în stocarea de fișiere
- Prezentarea tehnologiilor folosite în procesul de dezvoltare și a altor concepte cheie

1.2 Structura lucrării

Conținutul esențial al acestei lucrări este structurat în patru părți:

Prima parte este reprezentată de Capitolul 2 – Fundamente teoretice, unde vor fi prezentate aspecte mai pe larg despre mediile cloud, cum ar fi evoluția lor, arhitectură, modele oferite de furnizori, aplicații web bazate pe astfel de medii, dar și noțiuni referitoare la îmbunătățirea aplicațiilor din punctul de vedere al ușurinței de utilizare, ce trebuie avut în vedere în crearea de aplicații intuitive.

Următoarea parte, Capitolul 3 – Redundanța în sisteme distribuite, se focusează pe aprofundarea noțiunilor despre redundanță în cadrul sistemelor de fișiere, ce metode există, în special în cloud și vor fi prezentate de asemenea contribuțiile care pot fi aduse pentru utilizatori pornind de la soluții existente.

Partea a patra reprezentată de Capitolul 4 – Descrierea Aplicației, prezintă procesul de dezvoltare al aplicației. În prima parte sunt evidențiate funcționalitățile aplicației, precum și aspectul și comportamentul paginilor acesteia. Ulterior, este descris procesul de implementare, oferindu-se explicații pentru deciziile luate pe parcursul dezvoltării.

2 Fundamente teoretice

2.1 Tehnologii cloud

2.1.1 Scurtă prezentare

Așa cum este ilustrat în [6], conceptul de cloud computing a apărut pentru prima dată în jurul anilor 1960 sub forma tehnologiei de time-sharing, care oferea utilizarea în mod simultan a aceluiasi sistem de calcul de către mai mulți utilizatori. Având în vedere că în general ideea din spatele tehnologiei cloud este de a împărți resurse pentru cât mai mulți utilizatori din cadrul acelorași centre de date, putem spune că această tehnologie de time-sharing reprezintă un precursor al tehnologiei cloud. Cu toate acestea, abia în anul 1997, termenul de cloud computing este folosit pentru prima dată de către Ramnath Chellapa, cu scopul de a prezenta metoda prin care prin intermediul internetului se pot pune la dispoziție diferite resurse fizice. Progresul a continuat pentru această nouă idee, iar prin mijlocul anilor 2000 au apărut opțiuni de virtualizare și servicii web scalabile, apărând și primii mari furnizori de astfel de servicii precum Amazon Web Services (AWS), Microsoft Azure și Google Cloud Platform.

În [12], se prezintă o definiție a ceea ce înseamnă cloud computing ca fiind un model de calcul în care se oferă drept servicii diferite capacități IT cu un grad ridicat de scalabilitate. Acest lucru se realizează prin intermediul internetului, oferind o gamă largă de servicii precum capacitate de stocare, putere de procesare, aplicații de business sau componente.

Tehnologiile cloud se bazează pe mai multe proprietăți, printre care accesul de la distanță, elasticitate și scalabilitate, modelul de plată “pay-as-you-go” și resurse partajate. Pentru a dezvolta puțin, practic se asigură faptul că indiferent de locația fizică a resurselor, acestea pot fi folosite de oriunde există o conexiune la internet, fiind un mare plus pentru dezvoltatori în reducerea complexității gestionării unei rețele locale și doar concentrarea pe folosirea de resurse. În același timp, elasticitatea și scalabilitatea oferă utilizatorilor confortul de a ajusta într-un timp scurt capacitatea resurselor folosite în funcție de nevoi; astfel, dacă un server este mai mult

folosit într-o anumită perioadă, se oferă o capacitate mai mare de procesare prin creșterea de exemplu a numărului de mașini virtuale folosite ori a memoriei acestora. Modelul de plată folosit de tehnologiile cloud este unul la îndemână utilizatorilor, caracterizându-se prin faptul că se plătește fix ceea ce se consumă, acest lucru putând fi contorizat fie prin volumul de date procesate, timpul de utilizare al resurselor, capacitatea de stocare folosită, sau numărul de cereri efectuate către servicii. Cât despre resursele partajate, acestea se referă la faptul că infrastructura folosită pentru accesarea lor este comună utilizatorilor, dar resursele în sine sunt separate logic; un exemplu ar putea fi dat de memoria RAM alocată pentru mai multe mașini virtuale, fiind distribuită dinamic în funcție de situațiile apărute.

Cum nici o tehnologie nu este lipsită de potențiale provocări pe parcursul dezvoltării software, tehnologia cloud vine cu o serie de astfel de aspecte de luat în calcul, cum este prezentat în [6]. O primă problemă care se ridică este dată de siguranța datelor, care practic sunt înmânate unei alte entități, putând fi expuse diferitor atacuri cibernetice. Gestionarea costurilor este un alt aspect de considerat, în special în cazul companiilor mici care doresc să migreze soluțiile lor software în cloud; costurile pot ajunge în această situație destul de mari pentru bugetul lor, deși furnizorii de servicii cloud oferă și planuri separate pentru astfel de situații. O problemă majoră o consideră nevoia de specialiști în domeniu care cunosc într-un mod amănunțit serviciile și costurile implicate la care se expun, aceștia fiind de multe ori expuși la luarea de decizii din mai multe oferte posibile și trebuind să fie atenți la cea mai bună opțiune pentru situația lor. Nu în ultimul rând, trebuie avut în vedere și aspectul legal în respectarea integrității datelor, ce poate și ce nu poate fi păstrat în cloud și/sau trebuie stocat local.

2.1.2 Arhitectura cloud computing

Arhitectura cloud computing se referă la modalitatea prin care sunt organizate resursele și serviciile pe care mediile cloud le oferă spre utilizare. Cum este prezentat în [1][6][12], sunt trei tipuri în mod tradițional de gestionare a resurselor și serviciilor cloud: Infrastructure as a Service (IaaS), Platform as a Service (PaaS) și Software as a Service (SaaS). Diferența dintre modele, așa cum voi explica imediat, este dată de gradul de control pe care utilizatorul îl are asupra resurselor. Cum este de așteptat, un grad mai mare de control implică și o mai mare flexibilitate, dar și în același timp responsabilitate din partea dezvoltatorilor.

Infrastructure as a Service (IaaS) este nivelul cel mai de bază din arhitectura cloud în sensul în care utilizatorul poate opera la un nivel cât mai aproape de componentele fizice din infrastructura IT: resurse de procesare, stocare, rețea și alte astfel de componente hardware. Un aspect de bază din cadrul acestui model de organizare este dat de utilizarea virtualizării, prin care resursele fizice sunt abstractizate și împărțite în unități logice în cadrul de mașini virtuale (VM-uri). Acestea sunt izolate atât de partea hardware, cât și unele de altele, ce oferă securitate și flexibilitate, dar și un control ridicat asupra lor de către utilizatori. Putem observa de aici cum modelul IaaS este unul în care mai mulți clienți folosesc practic aceeași infrastructură fizică, fără a interacționa direct unul cu celălalt; ca și exemple de astfel de servicii, putem să ne gândim întru totul la furnizorii de cloud precum AWS ori Microsoft Azure, ce oferă VM-uri.

Platform as a Service (PaaS) reprezintă un nivel superior în schema serviciilor cloud în care se permite dezvoltarea, testarea și gestionarea aplicațiilor. Putem să ne gândim la PaaS în comparație cu IaaS ca la o infrastructură deja întreținută de către furnizorii de servicii cloud, utilizatorii având astfel la dispoziție un mediu în care își pot dezvolta aplicațiile fără grija întreținerii de servere și a altor aspecte mai de bază. În acest mod, se reduc costul și timpul de dezvoltare al aplicațiilor, dar nici dezvoltatorii nu se mai bucură de o flexibilitate totală asupra mediilor în care aplicațiile lor sunt găzduite. Acest model se pliază pentru proiecte în care focusul este pe o livrare cât mai rapidă a produsului ori pe aplicații distribuite, ca și exemple putând lua în considerare Heroku ori Microsoft Azure App Services în care practic se oferă medii de găzduit pentru aplicații web.

Software as a Service (SaaS) este nivelul cel mai de sus în care utilizatorul are acces doar la produsul final și practic nu gestionează într-un mod concret infrastructura pe care aplicația funcționează și nici nu se ocupă de dezvoltarea aplicației în sine. Astfel, utilizatorii vor folosi o instanță comună a aplicației, dar vor fi diferențe în datele și permisiunile acestora. Google Workspace ori Microsoft 365 reprezintă soluții de tip SaaS, în care așa cum am explicat, doar se oferă utilizatorilor un produs final, aceștia folosind totuși în mod comun resursele.

În articolul [12], mai este prezentat un model de arhitectură, *Data Storage as a Service (DaaS)*, ce este unul mai simplu, oferind servicii de stocare efectivă a datelor în medii cloud. Cu alte cuvinte, spațiul de stocare este virtualizat și accesibil când utilizatorii au nevoie de el. Un avantaj oferit de furnizorii de servicii cloud pentru acest tip de arhitectură în comparație cu soluțiile

oferite de servere locale este dat de scalabilitate și acces de la distanță chiar prin intermediul de API. Amazon S3, Azure Blob Storage și Google Cloud Storage sunt astfel de soluții specializate în stocarea de fișiere, dar sunt oferite și soluții pentru baze de date mai moderne, de tip tabelar sau No-SQL, precum Amazon RDS sau Firebase Realtime Database.

2.1.3 Modele de implementare în cloud computing

În funcție de modul în care sunt oferite și gestionate serviciile cloud, există mai multe modele de implementare. În sursa [1], sunt prezentate astfel 4 modele de implementare: Cloud Public, Cloud Privat, Cloud Comunitar și Cloud Hybrid.

Modelul *Cloud Public* presupune că serviciile sunt oferite de către un furnizor terț (precum Amazon Web Services) și sunt accesibile pentru oricine are nevoie să le folosească contra-cost, după modelul deja amintit de “pay-as-you-go”. Resursele sunt împărțite între mai mulți clienți, iar ca avantaje putem aminti costurile inițial reduse, scalabilitate rapidă și plasarea responsabilității mentenanței asupra furnizorilor. Cu toate acestea, există dezavantajul în limitarea personalizării infrastructurii.

În *Cloud Privat*, după cum și numele sugerează, infrastructura este dedicată în întregime unei singure organizații și poate fi găzduită fie intern ori de către un furnizor. În acest model, flexibilitatea este mai ridicată în luarea de decizii asupra infrastructurii, fiind vorba de control total asupra resurselor și de gestionarea proprie a problemelor de securitate, dar în același timp scalabilitatea este una mai redusă.

Cloud-ul comunitar este împărțit între mai multe organizații ce prezintă interese comune, cum ar fi instituțiile de cercetare ori organizațiile guvernamentale. Reprezintă un caz mai special de astfel de infrastructuri bazate pe tehnologia cloud și vine cu o serie de provocări distincte precum un acord clar între părți asupra modului de gestionare comun a resurselor și de mecanisme speciale de coordonare.

Ultimul model, *Cloud Hybrid*, este practic un model mai “liber” în care sunt combinate proprietăți din modelele menționate anterior. Scopul este bineînțeles de a întrebuința cât mai optim caracteristici din fiecare model prezentat anterior spre o soluție eficientă pentru problema

avută în vedere. Se urmărește de regulă păstrarea datelor într-un mediu privat și implementarea de sarcini mai puțin critice în mediu public.

2.2 Aplicațiile web

Aplicațiile web fac parte din viața noastră de zi cu zi datorită ușurinței cu care pot fi utilizate de către utilizatorii obișnuiți, care nu au nevoie de cunoștințe de programare, dar și datorită aplicabilității lor într-o varietate de domenii. Dezvoltarea aplicațiilor web a început în anii '90, când acestea erau alcătuite din pagini HTML statice. Ulterior, a fost posibilă adăugarea de fișiere media, însă paginile rămâneau în continuare statice. Un punct de cotitură în evoluția lor a fost apariția limbajului JavaScript în 1995, care a permis introducerea animațiilor și interacțiunii vizuale. Trecerea de la pagini statice la aplicații dinamice s-a concretizat odată cu lansarea tehnologiei Ajax (Asynchronous JavaScript and XML) în anul 2005. Așa cum este menționat în [7], aceasta a rezolvat una dintre principalele limitări ale aplicațiilor web tradiționale - utilizarea unui protocol sincron de tip cerere-răspuns, care genera întârzieri între pagini. În schimb, Ajax a permis trimiterea de cereri asincrone către server, menținând interfața utilizatorului activă și receptivă. De la introducerea tehnologiei Ajax, domeniul aplicațiilor web a cunoscut o dezvoltare accelerată, atât pe partea de server (backend), cât și pe cea de interfață (frontend). Aplicațiile web moderne se caracterizează prin flexibilitate, scalabilitate, interactivitate, fiind în același timp bazate pe cloud și compatibile cu diferite platforme.

2.2.1 Structură

Aplicațiile web dispun de o arhitectură specifică, structurată în mai multe diviziuni logice și fizice, denumite niveluri. Cea mai frecvent utilizată arhitectură este cea pe trei niveluri care include: nivelul de prezentare, nivelul aplicației și nivelul de date.

Principalul avantaj al acestei arhitecturi este faptul că fiecare nivel rulează pe o infrastructură separată, aspect esențial în procesul de dezvoltare. Astfel, fiecărui nivel i se poate alocă o echipă distinctă, iar toate echipele pot lucra în paralel. De asemenea, actualizările necesare într-un anumit nivel pot fi realizate independent, fără a afecta funcționarea celorlalte componente. Un alt

beneficiu important constă în faptul că utilizatorul nu are acces direct la baza de date, ceea ce oferă un grad sporit de securitate — deși, în continuare, pot apărea vulnerabilități.

Totuși, arhitectura pe trei niveluri nu este lipsită de dezavantaje. Printre principalele limitări se numără complexitatea mai ridicată față de arhitectura pe două niveluri, în care clientul comunică direct cu baza de date. În plus, aceasta implică un consum mai mare de timp și resurse — chiar și o modificare minoră necesită testări în mai multe componente ale sistemului.

Cum este prezentat în [13], o aplicație web distribuită are două părți principale: **partea client** și **partea server**, legătura dintre acestea fiind realizată prin intermediul canalului de comunicare dat prin **nivelul aplicației**.

Partea client se ocupă de interacțiunea dintre utilizator și calculator, având rolul de a transforma răspunsurile HTTP primite de la server într-o interfață grafică utilizabilă pentru utilizator. Aceasta este alcătuită din mai multe componente:

- *Nivelul de prezentare* – gestionează logica din partea de interfață și reprezintă codul nivelului superior din arhitectură.
- *Browser-ul* – interpretează codul straturii de prezentare pentru a-l afișa corespunzător și pentru a asigura comportamentul dorit.
- *Stocarea locală* – utilizată de nivelul de prezentare pentru a salva date local, prin mecanisme precum cookies, localStorage.
- *Sistemul de operare* – platforma pe care rulează browserul și, implicit, partea client din cadrul aplicației.

Nivelul aplicației reprezintă componenta centrală a arhitecturii, locul unde sunt procesate datele provenite fie din straturii de prezentare, fie din cel de date. Acest nivel poate fi considerat un intermediar între celelalte două straturi.

Partea server este compusă din următoarele componente:

- *Nivelul logic* – implementează logica de business a aplicației, utilizând un limbaj de programare de nivel înalt, precum Java sau C#.

- *Nivelul serverului web* – primește cererile HTTP din partea clientului, le interpretează și le transmite programului server-side corespunzător. Exemple de servere web includ Apache HTTP Server, Windows IIS și Nginx.
- *Stratul de stocare a datelor* – reprezintă baza de date propriu-zisă, locul unde este păstrată întreaga informație a aplicației.
- *Sistemul de operare* – este necesar pentru rularea tuturor celorlalte straturi din partea serverului.

O altă formă de arhitectură este cea cu două niveluri, bazată pe modelul client-server, în care logica aplicației este integrată în stratul de date. Astfel, comunicarea dintre client și server se realizează direct, ceea ce duce la o creștere a vitezei de execuție a aplicației. Totuși, această arhitectură prezintă anumite dezavantaje, printre care se numără lipsa de scalabilitate în cazul unui număr mare de utilizatori și nivelul redus de securitate, comparativ cu modelele pe trei niveluri.

2.2.2 Aplicațiile web în mediile cloud

Aplicațiile web bazate pe medii cloud sunt aplicații ce folosesc într-un mod consistent serviciile oferite de furnizorii de soluții cloud, atât de la diferite capacități precum putere de procesare, evenimente ce rulează la un anumit interval de timp până la găzduirea efectiv a aplicației. Aceste aplicații nu mai sunt pe un server fizic local, ci sunt distribuite în centre de date în diferite colțuri ale lumii, conectate prin internet. Utilizatorul are acces astfel la aplicația propriu-zisă în mod direct prin intermediul unui browser web, iar resursele oferite de către furnizori (stocare, servere, baze de date) sunt cele ce lucrează în spate pentru a menține aplicația în stare de funcționare.

În ceea ce privește arhitectura aplicațiilor găzduite în cloud, de cele mai multe ori apelează la variante moderne, bazate pe microservicii și containere, conferind un grad ridicat de flexibilitate și scalabilitate. Putem spune că respectă arhitectura pe trei nivele discutată anterior, prin faptul că cele două părți sunt separate și pot fi dezvoltate și scalate în paralel. Pot fi folosite diferite servicii oferite de către furnizori de tip Function as a Service (FaaS) (precum AWS Lambda ori Microsoft Azure Function) pentru eliminarea nevoii de servere fizice sau virtuale.

Un aspect important al aplicațiilor din medii cloud este dat de scalabilitate, acest lucru făcându-se în funcție de cerințele utilizatorilor. Sunt oferite în acest sens mecanisme de auto-scalare, menite fie să oprească ori să lanseze instanțe ale aplicației pe baza traficului ori a gradului de folosință a resurselor. De asemenea, astfel de aplicații pot fi distribuite geografic în mai multe regiuni iar cu ajutorul tehnicilor de load-balancing se realizează o distribuire eficientă a traficului.

În același timp, nu trebuie uitat avantajul dat de automatizarea procesului de livrare și implementare a codului prin CI/CD (Continuous Integration/Continuous Deployment). Soluții precum GitHub Actions sau Azure Pipelines oferă dezvoltatorilor mecanisme prin care pot configura diferite acțiuni când codul este modificat, cum ar fi validarea acestuia iar apoi modificarea aplicației din cadrul serverului pentru actualizarea acesteia într-un timp cât mai scurt.

Furnizorii de servicii cloud oferă pe partea de securitate și gestionarea identității servicii variate prin care se încearcă sporirea gradului de siguranță a aplicațiilor. Astfel, sunt oferite mecanisme prin care aplicațiile pot acționa precum o identitate separată cu un set de drepturi în conexiuni cu alte servicii, precum în apeluri către baza de date. Datele sunt criptate atât în tranzit cât și în repaus și există o gamă variată de firewall-uri și alte politici pentru protejarea aplicațiilor.

2.3 Practici în crearea de interfețe intuitive

În cadrul aplicațiilor web, un rol important în implementarea acestora este dat de modul în care interfețele vizuale sunt proiectate. Acest lucru este dat de simplul fapt că interfața grafică reprezintă primul (și în mare parte cel mai important) contact pe care utilizatorul îl are cu datele sale, cu ce îi este oferit spre a-i simplifica viața. O aplicație în care conținutul nu este unul ușor de înțeles sau nu există o consistență a design-ului reprezintă de la început un impediment în ușurința utilizatorului de a folosi acea aplicație, cu atât mai mult cu cât poate nu este unul obișnuit în domeniul tehnologiei informației.

În sursa [8], se oferă o perspectivă interesantă asupra rolului interfețelor grafice în cadrul aplicațiilor web, acela de a comunica cu utilizatorul, fiind o poartă de deschidere spre lumea datelor din spate. Se pune astfel accentul pe o comunicare eficientă în interacțiunea om-sistem, în

care scopul să nu fie doar în transmiterea de informații, ci și în mesaje clare și coerente pentru a facilita procesul de înțelegere al utilizatorului asupra acțiunilor pe care le întreprinde. În următoarele paragrafe voi prezenta câteva idei prezentate în cartea respectivă.

Un prim punct de plecare spre obținerea unui design intuitiv este dat de simplitate. Acest lucru nu constă în eliminarea de funcționalități, ci în prezentarea lor într-un mod cât mai clar, neaglomerat și ușor de înțeles. Se urmărește astfel evitarea situațiilor în care utilizatorul este suprasolicitat printr-o multitudine de butoane, opțiuni sau informații care nu aduc multă valoare, creând doar mai multă confuzie. Scopul general este ca printr-un design simplu, utilizatorul să se simtă practic ghidat în pașii pe care trebuie să îi întreprindă spre îndeplinirea unei acțiuni.

O altă idee pe care putem să o extragem din sursa citată este dată de importanța vizibilității și a ierarhizării informației. Este recomandat să se folosească o structură vizuală clară – fonturi mai mari sau culori contrastante pentru titluri sau link-uri spre alte pagini, butoane evidențiate pentru acțiuni esențiale și gruparea logică a elementelor legate între ele. Practic, dacă un utilizator este nevoit să caute prea mult care ar fi pasul următor într-o configurare de exemplu, atunci interfața nu este una intuitivă, iar acest lucru poate fi ușor de constatat în cazul interfețelor oferite de marii furnizori de servicii cloud în care doar cei experimentați în acest domeniu, ulterior pe baza unor certificate obținute, pot folosi mai ușor opțiunile interfeței grafice.

Un alt principiu important este oferirea de feedback clar și într-un timp scurt pentru acțiunile întreprinse de către utilizator. În acest caz, putem să ne gândim în cazul în care apar erori în urma unor acțiuni întreprinse de către utilizator, atunci ar trebui ca acestea să fie semnalate în mod corespunzător. Mai sunt alte astfel de situații precum aplicarea de animații în cazul în care un buton este apăsat sau afișarea de mesaje de încărcare, toate pentru a facilita o mai bună înțelegere a utilizatorului despre ce se întâmplă.

Nu în ultimul rând, autorul McKay propune un proces de design în care fiecare decizie este evaluată luând în considerare eficiența comunicării cu utilizatorul. Cu alte cuvinte, un pas important în crearea unui design intuitiv este dat de o bună înțelegere a nevoilor și așteptărilor utilizatorilor. Acest aspect, deși poate părea nesemnificativ la prima vedere, este extrem de important, fiind la baza ideii din spatele aplicației pe care o voi prezenta în unul din capitolele următoare: accentul trebuie pus pe capacitatea de înțelegere a utilizatorului asupra evenimentelor

pe care le întreprinde în cadrul aplicației, ce așteptări are de la aplicația pe care o va folosi și cum pot dezvoltatorii să răspundă acestor cerințe prin crearea unei interfețe grafice care să ofere confortul unei ghidări ușoare. Scopul final este ca utilizatorul să înțeleagă practic într-un timp scurt ce trebuie să facă pentru a realiza o acțiune pe care o dorește în cadrul aplicației.

Din articolul [5], se poate observa importanța personalizării interfețelor grafice prin crearea unei experiențe mai facile în utilizarea sistemului oferit. Se accentuează din nou ideea de centrare a focusului pe nevoile utilizatorilor, dar de această dată prin oferirea unei soluții care să fie pliabile pentru o gamă mai variată de cerințe prin adaptabilitatea design-ului. Pentru dezvoltatori, oferirea de opțiuni de personalizare nu înseamnă neapărat crearea de versiuni separate pentru grupuri diferite de utilizatori, ci mai degrabă oferirea de mijloace prin care aceștia își pot organiza după propriul plac interfața grafică.

3 Redundanța în sistemele distribuite

3.1 Principiile redundanței

Așa cum am mai precizat, redundanța se bazează pe crearea de mai multe copii de date pentru a asigura diferite proprietăți precum o mai bună accesibilitatea datelor, scalabilitate dar în principal conferă o siguranță de recuperare a datelor în cazul în care un nod, o zonă sau chiar o regiune eșuează.

Redundanța vine în același timp și cu unele probleme pentru dezvoltatori, cum ar fi costuri mai mari pentru stocare și lățime de bandă. Faptul că datele sunt ținute în mai multe regiuni implică practic o mai mare utilizare de resurse, de unde costul mărit dar și un potențial trafic mărit de rețea. De asemenea, trebuie luat în calcul și complexitatea adusă de sincronizarea datelor în special dacă se întâmplă frecvent, lucru ce poate afecta performanța.

O altă problemă majoră este dată de menținerea stării de consistență a datelor între replici. În principiu o replicare sincronizată a datelor conferă o mai mare consistență, dar vine și cu costuri mai ridicate. Aici sunt mai multe modele de consistență care se pot realiza, așa cum este prezentat în [2]:

- **Consistența slabă** permite cel mai scăzut nivel de garanții, ceea ce înseamnă că datele nu pot fi sincronizate și nu există nicio asigurare că sistemul va deveni vreodată consistent.
- **Consistența PRAM** asigură faptul că operațiile de scriere de la un singur proces apar în aceeași ordine pe toate replicile, dar ordinea scrierilor de la procese diferite poate varia.
- **Consistența cauzală** menține ordinea doar pentru operațiile care au o relație cauzală, ceea ce înseamnă că unele operații sunt văzute în aceeași ordine pe replici, dar nu toate.
- **Consistența secvențială** cere ca toate operațiile să apară în aceeași ordine pe toate replicile și ca operațiile de la un singur proces să fie executate în ordinea în care au fost emise.

- **Consistența strictă** oferă cea mai puternică garanție, cerând ca orice operație de scriere să fie vizibilă instantaneu pentru toate procesele, asigurând sincronizarea imediată pe toate replicile.

3.2 Tehnici cheie de redundanță

3.2.1 Replicarea datelor

Replicarea de date reprezintă o tehnică fundamentală în sistemele distribuite, în special în mediile cloud pentru motive în special legate de redundanță deja amintite. În special prin acest procedeu se asigură recuperarea datelor în caz de dezastru; chiar dacă un nod din sistem va eșua, astfel datele vor fi în continuare disponibile utilizatorilor, fiind necesar doar ca apelurile să fie redirecționate spre zona neafectată unde erau salvate datele.

Cum este menționat în [9][11], sunt în principiu 2 tipuri de metode de replicare:

- **Replicarea statică:** În această abordare, numărul de replici și locațiile acestora sunt predefinite și fixe. Deși această metodă este simplă și ușor de implementat, îi lipsește flexibilitatea și poate să nu se adapteze bine la modelele de acces sau condițiile de sistem în schimbare.
- **Replicarea dinamică:** Strategiile dinamice ajustează numărul și plasarea replicilor pe baza unor factori în timp real, cum ar fi frecvența accesării datelor, performanța nodurilor și condițiile rețelei. Această adaptabilitate permite o utilizare optimizată a resurselor și poate duce la o performanță mai bună și o eficiență a costurilor mai mare.

Un exemplu important din articolul [9] este dat de **Dynamic Data Replication Strategy (D2RS)**, propusă de Da-Wei Sun și colaboratorii. Această strategie se bazează pe un principiu simplu: de regulă, datele utilizate recent au un grad mai mare de probabilitate de a se folosi în viitorul apropiat; se remarcă prin utilizarea de parametri ca disponibilitatea sistemului, numărul de replici și consumul de lățime de bandă.

3.2.2 Redundanța geografică

Redundanța geografică constă în replicarea datelor în mai multe centre de date aflate în zone geografice diferite, dar mai important în acest caz este modul în care aceste zone sunt împărțite. În sistemele moderne cloud, spațiul geografic în care se află computere în care sunt stocate informații este împărțit abstract conceptual în zone sau regiuni, fiecare reprezentând o grupare de centre de date apropiate, dar izolate din punct de vedere al resurselor critice. De exemplu, o zonă poate fi formată din mai multe regiuni, zonele fiind la distanțe semnificativ mai mari între ele decât în cazul regiunilor. Cu cât datele sunt salvate în mai multe zone și regiuni diferite, cu atât este mai ușor de recuperat datele într-un dezastru ori de oferit o mai bună accesibilitate.

3.2.3 Mirroring-ul datelor

Mirroring-ul datelor reprezintă una dintre cele mai complexe metode de asigurare a redundanței în sistemele distribuite și în cloud. Pentru a înțelege mai bine ce implică mirroring-ul datelor, voi face o scurtă paralelă cu replicarea datelor pe care am dezbătut-o deja. Astfel, dacă cea de a doua modalitate este de obicei implementată asincron și orientată adesea spre scalabilitate și disponibilitate, mirroring-ul implică menținerea unei copii exacte, sincronizată într-un timp cât mai aproape de cel real, copie ce va fi stocată de cele mai multe ori într-un alt nod din rețea, fie că acesta este într-o altă zonă geografică sau nu. Scopul este de a avea la îndemână o copie a datelor care să poate fi folosită într-un mod imediat în caz de defecțiuni neașteptate, dezastru naturale etc.

De la o primă impresie, putem să ne dăm seama că această metodă se aplică de regulă în cazurile în care este important să avem la dispoziție imediat o copie a datelor, în special dacă acestea conțin informații sensibile; putem să ne gândim astfel la infrastructuri necesare în domeniile medical, militar ori financiar în care pierderea de date poate avea consecințe grave pentru cei ce folosesc sistemele în cauză. Mirroring-ul vine astfel în ajutor pentru a permite utilizatorilor să își continue activitatea în cazuri de probleme majore neprevăzute, fără grija că datele lor ar putea fi pierdute în acest proces (bineînțeles, nimic nu este sigur, dar probabilitatea este una extrem de redusă).

În articolul [10], autorii Patterson și Manley prezintă un sistem interesant de implementare a mirroring-ului asincron, SnapMirror, un lucru nu tocmai des întâlnit din cauza naturii acestei tehnici de redundanță. Astfel, dacă de obicei se utilizează metode de copiere a datelor în timp real, SnapMirror propune o soluție prin care datele sunt copiate la intervale regulate de timp, oferind un compromis între performanță și consistență.

La bază, SnapMirror se folosește de pe mecanismul de snapshot-uri implementat în sistemul de fișiere WAFL (Write Anywhere File Layout). Snapshot-ul în mod tradițional reprezintă o copie a fișierului la un anumit moment, fără ca această să mai sufere modificări ulterioare (se poate face o comparație cu o “poză” la acel moment a fișierului). Inițial, se va realiza o copie completă a fișierului pe destinație și se salvează un snapshot de referință. La următoarea sincronizare a celor două fișiere, se creează un nou snapshot la sursă, iar ingeniozitatea din spatele SnapMirror vine prin faptul că prin comparație dintre cele mai recente două snapshoturi de la sursă, se vor copia la destinație doar blocurile de date modificate. Avantajele acestui sistem constau în optimizarea consumului de lățime de bandă și într-o scădere a timpului necesar pentru sincronizare. Copiile create într-un alt nod din rețea sunt de regulă read-only, dar se pot modifica de asemenea cu posibilitatea de modificare în cazul în care fișierul principal nu mai este disponibil, iar copia devine fișierul de bază.

Mirroring-ul datelor vine de asemenea cu o serie de provocări. Una dintre acestea o constă dificultatea în menținerea sincronizării în timp real între două sau mai multe copii; acest lucru presupune un consum ridicat de resurse hardware și de rețea, ceea ce va implica mai departe costuri mărite. O altă problemă este dată de consistența datelor care poate apărea în momentele în care se creează copii; dacă apar erori în timpul acestui proces, copiile pot conține informații eronate, inconsistente, existând diferențe între original și replică. Modelul prezentat SnapMirror rezolvă o parte din aceste probleme prin replicarea asincronă, în acest caz timpul destinat copierii datelor fiind unul mai scăzut în care se copiază doar blocurile de date ce au suferit modificări, lucru ce va conduce la o șansă mai scăzută în apariția de erori dar și în copierea de date invalide.

3.3 Disponibilitate ridicată

Disponibilitatea ridicată reprezintă capacitatea unui sistem de a-și pune la dispoziție resursele în mod continuu și de a furniza servicii fără întreruperi semnificative, chiar și în cazuri în care pot apărea diverse malformații, fie erori software sau de cauză hardware. Mediile cloud, prin focusul lor de a pune la dispoziție utilizatorilor resursele într-o cât mai mare măsură de timp, au nevoie de implementări care să susțină o disponibilitate ridicată a datelor și implică mecanisme robuste de redundanță, toleranță la erori, recuperare rapidă după defecțiuni. Cum se poate observa în Tabelul 3-1, există o gamă variată de definiții exacte pentru ce reprezintă mai concret o disponibilitate ridicată a datelor. Se pot observa astfel câteva exemple, precum datele să fie accesibile 99.999% din timp, sau ca site-ul respectiv să nu fie disponibil doar pentru un maxim de 5 minute pe an.

Reference	Definition
Achieving High Availability at the Application Level in the Cloud [7]	The services provided by the applications are considered highly available if they are accessible 99.999 % of the time (also known as five 9's)
Managing Application Level Elasticity and Availability [25]	High availability is achieved when the outage is less than 5.25 minutes per year
Scheduling highly available applications on cloud environments [35]	High availability systems are characterized by fewer failures and faster repair times
Are clouds ready for large distributed applications? [36]	High availability is defined in terms of downtime that is the total number of minutes the site is unavailable for events lasting longer than 5 minutes over a 1-year period
Software aging in the eucalyptus cloud computing infrastructure: Characterization and rejuvenation [37]	Availability is defined as the ability of a system to perform its slated function at a specic instant of time.

Tabel 3-1 Definiții pentru disponibilitate ridicată [3]

Lucrarea [3] descrie mai multe modele de redundanță utilizate pentru a asigura o disponibilitate ridicată, așa cum sunt prezentate în cadrul Availability Management Framework (AMF):

- **Modelul 2N:** fiecare unitate activă din cadrul sistemului prezintă o replică standby, ce are ca scop preluarea sarcinilor unității principale. Este unul dintre cele mai des-întâlnite modele din cauza simplității sale și a unei implementări facile
- **Modelul N+M:** reprezintă o extensie a modelului anterior, permițând mai multe unități să fie active sau în standby, N reprezentând numărul de unități active iar M numărul unităților în stare de repaus, neexistând un element comun între aceste două mulțimi (o unitate e fie în stare activă ori de standby)
- **Modelul N-way:** asemănător cu N+M, însă în cazul acestui model, unitățile pot gestiona atât sarcini active cât și de standby, pentru instanțe diferite ale aplicațiilor
- **Modelul N-way active:** în cadrul acestui model toate unitățile sunt în stare activă, o instanță a aplicației putând fi alocată în același timp în mai multe unități

După cum este ușor de observat, modelul N-way oferă un grad mai mare de flexibilitate și de folosire a resurselor, dar din motive practice, tot modelul N+N este cel mai folosit pentru argumente deja oferite anterior, incluzând garanții clare privind disponibilitatea ridicată. În practică, asigurarea unei disponibilități ridicate nu se rezumă doar la folosirea unui anumit model pentru redundanță, dar implică crearea unui sistem robust ce prezintă de asemenea și mecanisme de monitorizare și utilizare în mod imediat a unei alte replici în cazuri speciale.

Articolul [4] abordează problema redundanței datelor din perspectiva eficienței și fiabilității. Autorii pun în discuție faptul că, deși redundanța este esențială pentru disponibilitate și recuperare în caz de defecțiune, excesul de replici implică costuri ridicate atât în spațiu de stocare, cât și în lățime de bandă pentru sincronizare. Se pornește de la observația că majoritatea sistemelor de stocare în cloud folosesc scheme de replicare de tipul 3-way, ceea ce implică practic că fiecare fișier este copiat integral de 3 ori. Acest lucru implică un consum mare de resurse pentru spațiu și rețea, devenind o soluție inefficientă pe termen lung.

Pentru rezolvarea acestor probleme, se propun metode avansate de codare și replicare care să reducă numărul de date duplicate, însă fără a afecta fiabilitatea. Unele dintre soluții implică scheme de codare a erorilor, ori replicare selectivă, care permit recuperarea datelor deși unele replici nu mai sunt valabile, dar acestea vin cu dezavantajul unei capacități de stocare mai reduse. Modelul de fiabilitate propus în articol definește în mod formal relația dintre rata de eșec a nodurilor, nivelul de redundanță și probabilitatea de pierdere a datelor. Această abordare permite cuantificarea matematică a beneficiilor aduse de strategiile alternative față de replicarea clasică. De asemenea, soluția propusă ajustează parametrii codurilor de ștergere în funcție de condițiile sistemului (frecvența de acces, disponibilitatea nodurilor, nivelul de încărcare al rețelei etc.), datele critice beneficiind de o protecție mai ridicată.

3.4 Soluții existente

3.4.1 Redundanța în Azure

Azure prezintă o gamă variată de strategii de redundanță care operează pe mai multe nivele: infrastructură fizică (centre de date), nivel logic (zone și regiuni de disponibilitate) și nivelul serviciilor de stocare (Storage Account Redundancy). La nivel fizic, datele sunt replicate automat în funcție de opțiunile selectate de către utilizator, lucru ce înseamnă că, în cazul unor erori neprevăzute, datele vor rămâne accesibile prin intermediul replicilor.

Azure oferă următoarele opțiuni de redundanță la nivelul serviciilor de stocare (opțiunile fiind prezentate în ordinea crescătoare a costurilor dar descrescătoare a gradului de durabilitate oferit):

- **Locally Redundant Storage (LRS)** – sunt păstrate mai multe copii ale fiecărui obiect în același centru de date din cadrul regiunii de disponibilitate selectate de către utilizator
- **Zone-Redundant Storage (ZRS)** – datele sunt replicate în minim trei zone de disponibilitate diferite din cadrul aceleiași regiuni; în Azure, conceptul de regiune de disponibilitate nu se limitează la un singur centru de date, existând mai multe zone de disponibilitate în cadrul său

- **Geo-Redundant Storage (GRS)** – pe lângă replicile salvate în cadrul aceleiași regiuni de disponibilitate folosind LRS, mai sunt replici într-o regiune diferită de disponibilitate, replicate local în a doua regiune de redundanță tot folosind LRS; o regiune de disponibilitate include mai multe zone, delimitarea dintre acestea fiind mult mai clară, la nivelul sutelor de kilometri
- **Read-Access Geo-Redundant Storage (RA-GRS):** extinde GRS prin permiterea accesului în mod read-only la copia din regiunea secundară, oferă o mai bună accesibilitate și un timp mai mic de indisponibilitate în cazul în care regiunea primară nu mai funcționează corect

Fiecare zonă de disponibilitate din cadrul unei regiuni este independentă din punct de vedere al sursei de energie, rețea și răcire, în cadrul unei regiuni existând mai multe astfel de zone. Pentru aplicațiile critice, Microsoft recomandă implementarea unei arhitecturi care folosește atât zone de disponibilitate, cât și replicare geo-redundantă.

Pentru a optimiza costurile și performanța în funcție de frecvența de acces a datelor, Azure oferă trei Access Tiers (niveluri de acces), esențiale în gestionarea eficientă a datelor:

1. **Hot Tier** – destinat datelor accesate frecvent. Este cel mai avantajos din punct de vedere al costului de acces la date (read/write), dar cu un cost mai ridicat de stocare. Este ideal pentru date active, folosite zilnic, cum ar fi fișierele utilizate de aplicațiile curente sau de utilizatori finali.
2. **Cool Tier** – soluție intermediară, potrivit pentru datele accesate mai rar, dar care trebuie păstrate pentru o perioadă mai lungă de timp. Costul de stocare este mai redus decât în *Hot Tier*, însă operațiunile de acces sunt mai costisitoare. Acest nivel este recomandat pentru backup-uri periodice sau date istorice utilizate ocazional.
3. **Archive Tier** – optim pentru datele care trebuie păstrate pe termen lung, dar sunt accesate foarte rar (ex. la câteva luni sau ani). Este cel mai ieftin nivel în ceea ce privește stocarea, dar are un cost ridicat și un timp mare de latență la restaurare.

3.4.2 Redundanța în AWS

AWS implementează un model de redundanță robust și distribuit pe mai multe niveluri, fiind proiectat pentru a oferi disponibilitate ridicată, toleranță la erori și recuperare în caz de dezastru pentru aplicațiile și datele utilizatorilor. Arhitectura de bază a AWS este construită în jurul conceptelor de regiuni geografice și Zone de disponibilitate (Availability Zones – AZs). O regiune AWS este o locație fizică globală, în timp ce o zonă de disponibilitate este un set de centre de date independente dintr-o regiune, cu alimentare, rețea și răcire separate. Zonele sunt conectate între ele prin rețele de mare viteză, permițând replicarea rapidă a datelor și comunicarea între instanțe în mod stabil.

În mod asemănător cu procesul de replicare descris în cadrul sistemelor de la Microsoft Azure, AWS de asemenea oferă mai multe opțiuni în ceea ce privește redundanța datelor. În cazul S3 Standard, o opțiune similară cu ZRS oferit de Microsoft Azure, datele sunt replicate în minimum trei facilități dintr-o regiune AWS, ceea ce oferă o durabilitate de 99,999999999% (11 nouă) și disponibilitate ridicată. Pentru aplicații cu cerințe și mai stricte, Amazon S3 Cross-Region Replication (CRR) permite replicarea automată a obiectelor S3 în regiuni diferite, oferind o protecție avansată împotriva dezastrelor la scară regională.

În plus, AWS pune la dispoziție instrumente avansate pentru disponibilitate și control al traficului. Serviciul Elastic Load Balancing (ELB) distribuie automat traficul de aplicație între mai multe instanțe distribuite în zone de disponibilitate diferite. Route 53, serviciul de DNS al AWS, permite configurarea de politici de routing bazate pe locație, latență sau stare de sănătate, direcționând traficul către cele mai apropiate și funcționale resurse. Pentru recuperare în caz de dezastru, AWS Backup, împreună cu AWS Disaster Recovery și soluții personalizabile precum CloudFormation pentru refacerea automată a infrastructurii, permit organizațiilor să își refacă rapid mediul în alte regiuni în cazul unei defecțiuni majore. Prin combinarea acestor servicii și practici, AWS permite construirea de sisteme elastice, care răspund automat la incidente și perturbări, maximizând astfel uptime-ul și disponibilitatea aplicațiilor.

4 Descrierea aplicației

După cele prezentate în capitolele de teorie, voi realiza o prezentare a aplicației venite să ajute utilizatorii neantrenați în interacțiunea cu mediile cloud, dar care își doresc un control mai mare asupra modului în care sunt stocate fișierele acestora și a locului în care acest lucru se întâmplă.

Aplicația este una găzduită în Microsoft Azure, fiind implementată ca o aplicație web clasică într-un mediu cloud, beneficiind de CI/CD (Continuous Integration /Continuous Deployment) oferit de către GitHub. S-au folosit astfel mai multe servicii cloud precum App Service, Storage Account, Azure Function și Azure SQL. Funcționalitățile acestora sunt venite să ofere într-un design intuitiv aceleași capabilități precum cele întâlnite în Microsoft Azure, dar cu o configurare mult mai ușoară pentru utilizatorii mai puțin familiari cu această platformă. Ca element de noutate propriu se remarcă crearea unui model de redundanță în care utilizatorul va selecta două regiuni de disponibilitate în care dorește să își salveze fișierele, iar aplicația va gestiona singură mai multe elemente precum dispunerea celei mai recente versiuni valabile în acel moment, asigurarea consistenței dintre cele două copii prin verificare continuă a timpilor de modificare pentru acestea (și modificare în caz de diferențe).

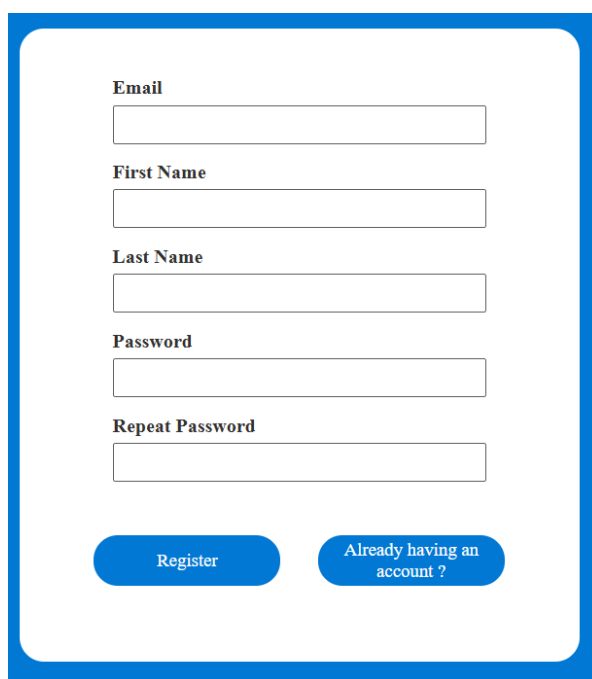
Astfel, designul mai simplu oferit pentru utilizatori va fi însoțit și de această oportunitate pentru utilizatori de a selecta individual cele două regiuni de redundanță în care doresc ca fișierul sursă să fie salvat. De asemenea, aplicația oferă și suport pentru previzualizarea fișierelor (în limita unor tipuri cunoscute precum imagini, documente text, pdf-uri sau documente din sfera Microsoft 365) în cadrul aplicației, scutind utilizatorii din a fi nevoiți să download-eze fișierele dacă doar vor un acces rapid la conținutul acestora. Se oferă și posibilitatea de versionare, în care utilizatorii pot salva mai multe versiuni pentru un fișier, față de Microsoft Azure putând denumi aceste versiuni pentru o mai bună gestionare a lor, și chiar compara versiuni între ele în cadrul aplicației prin previzualizarea conținutului acestora. Practic, aplicația vine cu majoritatea operațiilor necesare în lucrul cu fișiere pentru un utilizator simplu, dar care dorește în același timp să aibă un mai mare control asupra durabilității stocării fișierelor.

4.1 Interfața grafică și mod de utilizare

O caracteristică întâlnită în majoritatea aplicațiilor web este crearea de conturi pentru fiecare utilizator. Astfel, prima dată când o persoană dorește să utilizeze aplicația noastră, va fi afișat un ecran de autentificare, așa cum se poate observa în Figura 4-1. Dacă utilizatorul respectiv a mai accesat anterior aplicația și sesiunea sa nu a expirat, atunci acest ecran va fi înlocuit cu conținutul aplicației. Am setat o sesiune să dureze 24 de ore, ceea ce înseamnă că, dacă utilizatorul nu a folosit aplicația timp de mai mult de această perioadă, va trebui să se autentifice din nou.

The login form is enclosed in a blue border. It features two input fields: 'Email' and 'Password'. The 'Password' field includes a small eye icon for toggling visibility. Below the fields are two blue buttons: 'Log in' and 'Create a new account'.

Figură 4-1

The registration form is enclosed in a blue border. It contains five input fields: 'Email', 'First Name', 'Last Name', 'Password', and 'Repeat Password'. At the bottom, there are two blue buttons: 'Register' and 'Already having an account ?'.

Figură 4-2

Dacă utilizatorul nu are deja un cont, acesta poate apăsa butonul „Create a new account”, care îl va redirecționa către ecranul de înregistrare. Așa cum se poate observa în Figura 4-2, sunt solicitate câteva informații generale, precum numele utilizatorului, un email care trebuie să fie unic și o parolă care trebuie introdusă de două ori pentru confirmare. După ce înregistrarea se realizează cu succes, utilizatorul va fi redirecționat spre pagina de autentificare.

După ce utilizatorul se va autentifica cu succes, ecranul principal se va afișa precum se vede în Figura 4-3 unde vor fi afișate toate fișierele pe care le-a salvat până în acel moment, sub forma

unui tabel cu următoarele informații: Nume, Locația principală de stocare, Locația secundară (pentru fișierele ce dispun de redundanță Custom oferită de această aplicație), Redundanță, Suportul pentru versionare și Data creării.

File System AppCosmin Holcan

[+ Add File](#)
[Logout](#)

File name	Primary Location	Seconday Location	Redundancy	Supports Versioning	Creation Date
goodVersioning.pdf	West Europe		Locally	Yes	07.05.2025
fileVersioning.txt	West Europe		Locally	Yes	07.05.2025
Disertatie.docx	West Europe		Locally	Yes	16.05.2025
File.docx	North Europe		Zone	No	05.06.2025
TestZone.pdf	Germany West Central	West Europe	Custom	No	05.06.2025

Figură 4-3

După cum se observă din imagine, coloanele de Nume și cea de Suport pentru versionare (în cazul în care această opțiune este valabilă) sunt de fapt niște link-uri spre alte pagini, lucru evidențiat prin design-ul consacrat pentru acest tip de redirecționări. În dreapta sus se regăsesc două butoane, unul pentru opțiunea de adăugat un nou fișier, și cealaltă pentru ieșirea din aplicație de către utilizator.

În Figura 4-4, se observă componenta pentru adăugarea unui nou fișier cu opțiuni clare pentru utilizator în comparație cu procesul mai anevoios de creare a unui fișier în Microsoft Azure (în caz în care nu există deja un Storage Account cu toate setările deja făcute). Cum pentru un utilizator obișnuit îl interesează doar scopul final, acela de a crea fișierul cu setările dorite, acest meniu de configurări este unul mai simplu și intuitiv de folosit. Putem observa de asemenea și utilizarea de mesaje informative, concise și clare, pentru a îndruma utilizatorul în alegerea celei mai bune opțiuni pentru interesele sale.

Se regăsesc următoarele opțiuni pentru adăugarea de fișiere – nume (extensia e adăugată la final automat pe baza fișierului selectat local), zona de disponibilitate, tipul de redundanță (Locally, Zone, Globally sau Custom), a doua zonă de disponibilitate (acest al doilea dropdown este vizibil doar în cazul în care redundanța selectată este cea Custom iar opțiunile de selecție nu o includ pe cea de la zona primară de stocare), selectarea fișierului local, suportul pentru versionare (dacă

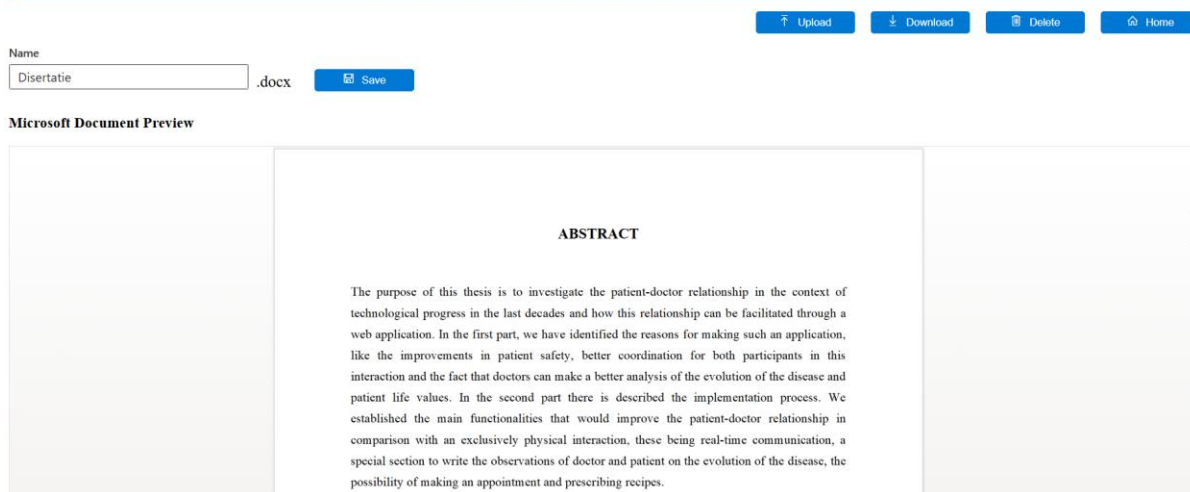
această opțiune este bifată, atunci utilizatorul va trebui să specifice și un nume pentru versiunea de bază, câmp precompletat cu opțiunea Original). Dacă tot procesul funcționează corect, atunci fișierul va fi adăugat în lista cu celelalte fișiere ale utilizatorului iar componenta se va închide, altfel un mesaj de eroare va fi afișat.

The screenshot shows a web form titled "Storage Redundancy Options". On the left side, there are four input fields: "Name" (a text box), "Location" (a dropdown menu), "Redundancy" (a dropdown menu), and "File" (a "Choose File" button with the text "No file chosen" next to it). Below the "File" field is a checkbox labeled "Support versioning". At the bottom left is a blue "Save" button with a floppy disk icon. On the right side, under the title, there are four storage options: "Locally Redundant Storage (LRS)" with a description, "Zone-Redundant Storage (ZRS)" with a description, "Geo-Redundant Storage (GRS)" with a description, and "Custom Redundancy" with a description.

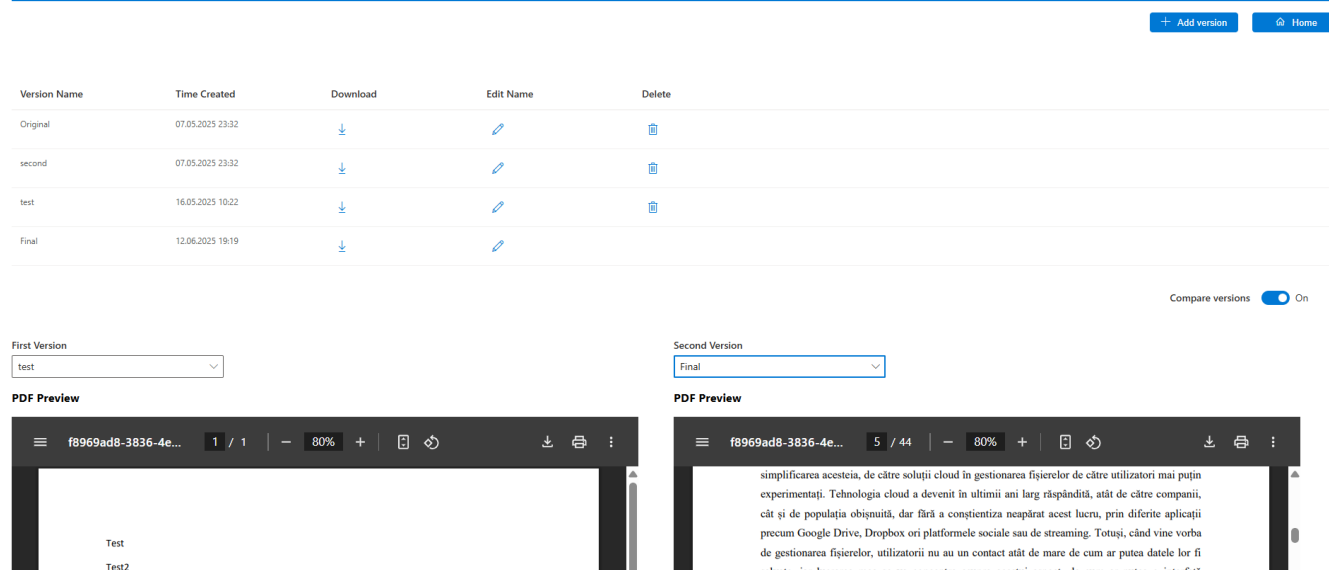
Figură 4-4

Pagina pentru fiecare fișier în parte (Figura 4-5) va conține următoarele elemente: previzualizarea fișierului și opțiuni pentru modificarea fișierului (se va deschide un modal asemănător cu cel prezentat anterior dar simplificat), download și întoarcerea spre pagina de pornire.

În ceea ce privește pagina pentru versionare (Figura 4-6), aceasta conține o listă cu toate versiunile pentru acel fișier, numele lor, când au fost adăugate și o serie de opțiuni pentru fiecare versiune în parte: . În partea dreapta sus, se vor regăsi butoanele pentru adăugarea unei noi versiuni, ce va deschide un nou modal pentru aceasta, și un buton de întoarcere spre pagina cu toate fișierele aceluși utilizator. De asemenea, se poate observa și un buton de toggle care va face posibilă fie compararea de două versiuni între ele prin selectarea acestora, fie doar selectarea unui singur fișier spre previzualizare.



Figură 4-5



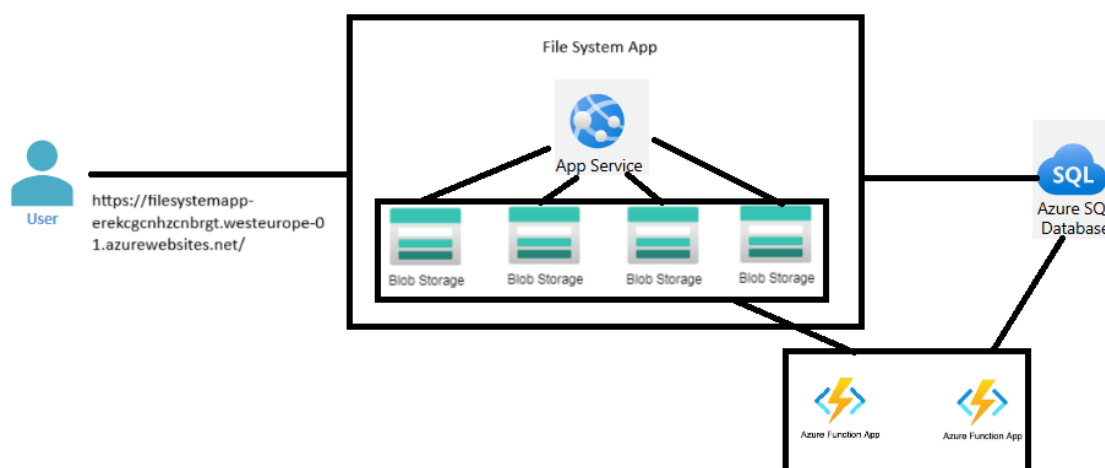
Figură 4-6

După cum se poate vedea din imagini, am încercat să respectăm cât mai mult indicațiile privind o interfață grafică intuitivă, elemente amintite în subcapitolul 2.3. Cum era descris acolo, scopul interfeței grafice este de a crea un canal de comunicare pentru utilizator, în care i se prezintă într-un mod clar care sunt pașii spre îndeplinirea unei operații din cadrul aplicației. Design-ul oferit de această aplicație este unul simplu față de cel oferit de marii furnizori de servicii cloud, în care sunt prezentate doar opțiuni de care un astfel de utilizator ar avea nevoie precum stabilirea regiunii ori a modului de replicare, fără alte detalii care nu l-ar interesa. Astfel, i se oferă opțiuni

de configurare și versionare a fișierelor într-un mod clar, fără a fi nevoie de pași intermediari, în cazul în care o operație eșuează atunci se vor afișa mesaje de eroare și de asemenea design-ul oferă posibilități de personificare prin elemente ca butonul de toggle din pagina de versionare care îi oferă utilizatorului posibilitatea fie de a previzualiza o sigură versiune a fișierului ori de a compara două versiuni între ele.

4.2 Arhitectura

În prima parte voi vorbi despre arhitectura cloud a aplicației, ce servicii s-au folosit și cum sunt conectate între ele, după care despre arhitectura software a soluției în sine.



Figură 4-7

În Figura 4-7 este reprezentată o schemă a serviciilor cloud folosite în cadrul aplicației. Astfel, inițial utilizatorul va accesa aplicația prin cadrul url-ului specificat direct din browser-ul său. Cererea sa va ajunge în cadrul aplicației care este găzduită cu ajutorul unui singur App Service (acest serviciu este folosit pentru funcționarea părții de back-end și front-end, doar că pentru a economisi resurse s-a folosit un singur astfel de serviciu, fișierele client fiind practic construite și apoi folosite de către aplicație).

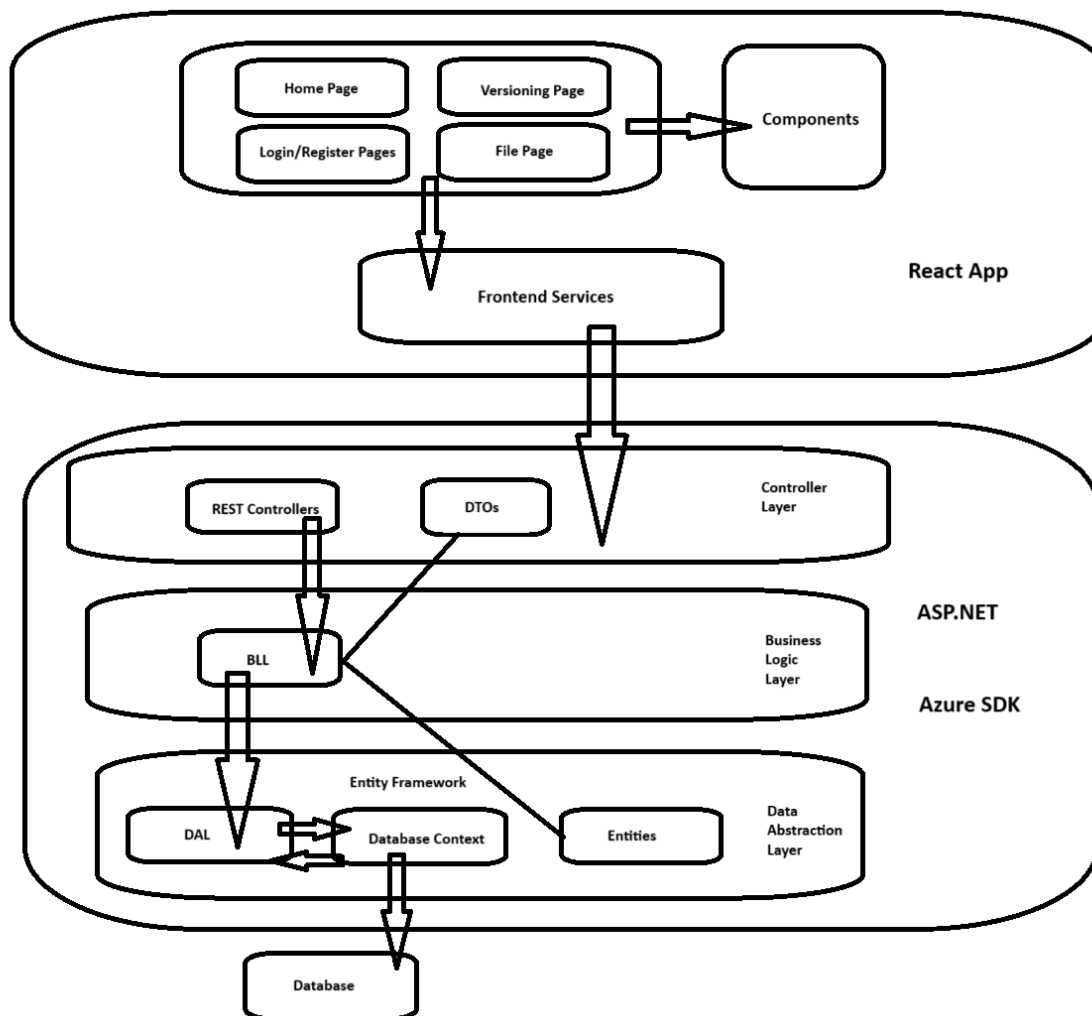
Fișierele vor fi stocate în mai multe Azure Storage Accounts, fiecare cu setări separate, decizia de stocare a unui fișier într-un astfel de spațiu fiind determinată de setările utilizatorului pentru fișier (regiune de disponibilitate, tip de redundanță, posibilitate de versionare). Informațiile privitoare la fișiere, utilizatori, versiuni de fișiere, etc. vor fi stocate tot în mediu cloud prin intermediul unei baze de date Azure SQL Database. Aplicația va face cererile către baza de date printr-o conexiune bazată pe un string de conectare și o parolă.

Vor fi folosite de asemenea și două Azure Functions, una pentru a verifica săptămânal cât de des sunt accesate fișierele pentru a le modifica tipul de acces din motive de cost, iar cealaltă pentru a verifica din oră în oră dacă fișierele pereche ce fac parte din opțiunea custom de redundanță oferită de aplicația noastră sunt sincronizate (în caz contrar, se va încerca modificarea conținutului copiei invalide și modificarea datelor corespunzătoare în cadrul bazei de date).

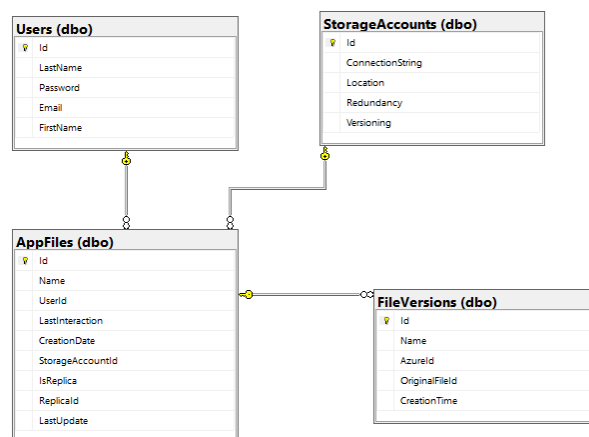
Figura 4-8 reprezintă arhitectura aplicației în sine. Se observă ușor că este o aplicație cu o parte de back-end implementată folosind tehnologii ASP.NET și o parte de front-end realizată cu ajutorul React.

Partea de back-end este compusă din trei nivele: nivelul de control (Controller Layer), nivelul de logică (Business Logic Layer) și nivelul de abstractizare a datelor (Data Abstraction Layer), fiecare având responsabilități specifice care vor fi discutate în continuare. Partea de front-end are o structură diferită, specifică aplicațiilor React. Fiecare pagină este, de fapt, o componentă funcțională care poate utiliza și componente mai mici. În acest mod, sunt create structuri funcționale care pot fi reutilizate cu ușurință. Vom explica mai multe despre aceste aspecte mai târziu, concentrându-ne acum pe structura back-end.

Data Abstraction Layer (DAL) este responsabil pentru comunicarea cu baza de date. Realizat cu ajutorul Entity Framework, reprezintă locul în care sunt create entitățile ce vor fi folosite de-a lungul aplicației și care vor fi transpuse în tabele în baza de date. Modalitatea aleasă pentru crearea bazei de date este code-first, ce implică mai întâi scrierea de cod pentru modele și apoi prin intermediul Entity Framework, sunt create tabelele. Pentru modificarea structurii tabelor se folosesc migrări, cod generate de Entity Framework între două stări ale bazei de date cu modificările necesare. În Figura 4-9 se observă schema bazei de date.



Figură 4-8



Figură 4-9

Business Logic Layer (BLL) are ca scop execuția regulilor specifice de business ale aplicației, fiind de fapt un strat intermediar între Controller Layer și Data Abstraction Layer. Practic, în cadrul acestui nivel sunt preluate cereri de Controller Layer, executat cod specific regulilor de business ale aplicației, interpretarea datelor abstracte din Data Abstraction Layer și transformarea acestora în modele de tip DTO ce vor fi transmise mai departe.

Controller Layer (CL) se ocupă cu procesarea cererilor venite din partea interfeței frontend, apelând funcțiile necesare din Business Logic Layer. Pentru fiecare entitate avem o clasă controller separată. Atunci când primim o cerere care implică operații pentru un utilizator deja autentificat, dorim să ne asigurăm că acel utilizator este cu adevărat conectat în aplicație. Astfel, pentru fiecare utilizator se creează un token JWT în momentul autentificării, iar ulterior acest token va fi utilizat ori de câte ori utilizatorul face o cerere.

În cele ce urmează, vom discuta despre implementarea părții de **Front-end**. La început, a fost implementat un Router care redirecționează utilizatorul către pagina de autentificare în cazul în care acesta nu este logat sau sesiunea a expirat. Pentru a determina acest lucru, se preia token-ul JWT din Local Storage și se încearcă reîmprospătarea acestuia. Dacă nu există un token în Local Storage sau dacă operația de reîmprospătare primește un răspuns de tip BadRequest de la server, atunci va fi afișată pagina de Login. În aceste cazuri, utilizatorul nu poate accesa conținutul aplicației până când nu se autentifică cu succes.

În ceea ce privește implementarea paginilor, este important de menționat că mai întâi au fost create componente funcționale mai mici, care au fost mai târziu reutilizate în componentele de tip pagină. Pentru a asigura o structură clară și organizată a întregului proiect, fiecare componentă are trei fișiere asociate: unul pentru stilizare cu sufixul .styles.ts, unul care conține definițiile tipurilor utilizate, având sufixul .types.ts, și unul pentru implementare, cu extensia .tsx

4.3 Implementare

În această parte a lucrării voi încerca să explic cele mai interesante aspecte din cadrul întregului proces de dezvoltare al aplicației, în special ce oferă această soluție diferit pe partea de redundanță, oferind exemple de cod unde este cazul.

Voi începe cu partea de CI/CD. Cum aceasta este o aplicație cloud, o soluție multumitoare pentru păstrarea codului într-un loc sigur și opțiunea de a modifica conținutul aplicației găzduite în cloud într-un timp cât mai scurt a fost dată de folosirea GitHub, în special pe partea de GitHub Workflow. Acesta ne-a permis crearea de pași automați în procesul de deploy al aplicației, practic la fiecare operație de push pe care o făceam pe repo-ul de GitHub fiind urmate aceste instrucțiuni. Astfel, sunt specificate mai multe informații precum distribuția mașinii pe care codul va rula, se face cache la pachetele din node modules deja existente, iar aplicația client este construită și folosită ulterior de partea back-end ca fișiere statice. O parte din acești pași pot fi văzuți în Figura 4-10.

```
# Step to cache client-side npm dependencies
- name: Cache npm dependencies client
  uses: actions/cache@v3
  with:
    path: client/node_modules
    key: ${{ runner.os }}-npm-client-${{ hashFiles('client/package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-npm-client-

# Setup Node.js
- name: Setup Node.js
  uses: actions/setup-node@v3
  with:
    node-version: '20'

# Build React app
- name: Build React app
  working-directory: ./client
  run: |
    npm install
    CI=false npm run build

# Ensure wwwroot directory exists inside the API project
- name: Ensure wwwroot directory exists
  run: mkdir -p ./server/server/wwwroot

# Clear existing frontend files in wwwroot directory (optional)
- name: Clear existing frontend files
  run: rm -rf ./server/server/wwwroot/*

# Copy React build files to wwwroot directory
- name: Copy React build files to wwwroot
  run: cp -r ./client/build/* ./server/server/wwwroot/
```

Figură 4-10

O altă problemă pe care am întâmpinat-o pe partea de CI/CD a fost cum ar trebui menținută baza de date la zi cu modificările pe care modelele le-au suferit ulterior. Soluția a fost una destul de simplă, fiind necesar ca în codul de start pentru aplicație să se verifice mai întâi dacă sunt migrări care nu au fost încă aplicate pe baza de date, iar dacă acesta este cazul, migrările respective să fie integrate, prin modificarea bazei de date în mod corespunzător. Migrările în Entity Framework reprezintă un mecanism prin care modificările aduse modelului de date (clasele C#) sunt

sincronizate cu schema bazei de date. Practic, ele permit actualizarea structurii bazei de date fără a pierde datele existente.

În DAL, specific aplicațiilor ce folosesc Entity Framework, se regăsește o clasă pentru contextul bazei de date `FileSystemAppDbContext` (Figura 4-11), în care sunt specificate entitățile folosite și care se vor regăsi în baza de date sub formă de tabele. În rest în cadrul DAL se regăsesc câte o clasă pentru fiecare entitate și câte o clasă care se va ocupa de instrucțiuni de bază de tip CRUD pentru aceste entități.

```
public class FileSystemAppDbContext : DbContext
{
    0 references
    public FileSystemAppDbContext() { }
    0 references
    public FileSystemAppDbContext(DbContextOptions<FileSystemAppDbContext> options) : base(options) { }
    4 references
    public DbSet<User> Users { get; set; }
    8 references
    public DbSet<AppFile> AppFiles { get; set; }
    2 references
    public DbSet<FileVersion> FileVersions { get; set; }
    4 references
    public DbSet<StorageAccount> StorageAccounts { get; set; }

    0 references
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<AppFile>()
            .HasMany(f => f.Versions)
            .WithOne(v => v.OriginalFile)
            .HasForeignKey(v => v.OriginalFileId)
            .OnDelete(DeleteBehavior.Cascade);

        base.OnModelCreating(modelBuilder);
    }
}
```

Figură 4-11

Structura directoarelor pentru BLL și CL este asemănătoare, existând câte o clasă separată pentru fiecare entitate în parte la fiecare nivel. De asemenea, o clasă de la un nivel superior poate interacționa cu clasele a mai multe entități de la un nivel inferior (de exemplu, `FileVersionsBLL` se folosește atât de `FileVersionsDAL` cât și de `AppFilesDAL`). Ce este mai interesant de menționat pe această parte este modul în care aceste clase sunt inițializate cu câte un obiect pentru fiecare clasă de nivel inferior de care au nevoie. Astfel, în fișierul `Program.cs`, ce este utilizat pentru configurarea serviciilor ASP.NET, sunt create mai multe obiecte cu ciclul de viață scoped pentru fiecare astfel de clasă specifică unei entități (Figura 4-12). Se înregistrează clasele în sistemul de Dependency Injection al aplicației, astfel încât o nouă instanță a clasei să fie creată pentru fiecare cerere HTTP. Aceasta permite injectarea automată a clasei acolo unde este

necesară (de exemplu, în constructorul clasei FileVersionsBLL, se vor injecta automat două obiecte corespunzătoare claselor FileVersionsDAL și AppFilesDAL, Figura 4-13).



```
builder.Services.AddScoped<UsersDAL>();
builder.Services.AddScoped<AppFilesDAL>();
builder.Services.AddScoped<StorageAccountsDAL>();
builder.Services.AddScoped<FileVersionsDAL>();

builder.Services.AddScoped<UsersBLL>();
builder.Services.AddScoped<AppFilesBLL>();
builder.Services.AddScoped<StorageAccountsBLL>();
builder.Services.AddScoped<FileVersionsBLL>();
```

```
public class FileVersionsBLL
{
    private FileVersionsDAL _fileVersionsDAL;
    private AppFilesDAL _appFilesDAL;

    0 references
    public FileVersionsBLL(FileVersionsDAL fileVersionsDAL, AppFilesDAL appFilesDAL)
    {
        this._fileVersionsDAL = fileVersionsDAL;
        this._appFilesDAL = appFilesDAL;
    }
}
```

Figură 4-12

Figură 4-13

Pentru partea de previzualizarea a fișierelor s-au creat tokenuri SAS care vor fi folosite pe partea de front-end. Un token SAS (Shared Access Signature) este un link temporar și securizat care permite acces controlat la resurse din Azure Storage, precum fișierele stocate de aplicația noastră. Această procedură oferă un grad mai mare de securitate, luând în considerare și faptul că au fost configurate drept private containerele din Azure Storage (lucru ce înseamnă că fișierele nu sunt accesibile doar printr-un simplu URL, fiind nevoie de astfel de token-uri).

În principal, codul este unul destul de intuitiv pentru necesitățile aplicației pe partea de back-end, dar ar trebui explicat în detaliu cum s-a realizat implementarea redundanței de tip custom pe care o propune această aplicație, fiind elementul de noutate adus față de marii furnizori de servicii cloud (exceptând partea grafică care este diferită și vine în ajutorul utilizatorilor neantrenați). Redundanța custom pe care o propun implică posibilitatea utilizatorului de a selecta două zone de disponibilitate în care dorește ca fișierele sale să fie stocate.

În primul rând, trebuie să stabilim că replicarea LRS oferă siguranța faptului că știm că datele vor fi salvate într-o anumită regiune, iar datele sunt replicate măcar în cadrul aceleiași centru de date. Putem astfel să pornim de la acest model pentru ca utilizatorul să știe exact în ce zone fișierele sunt replicate.

Modelul propus pentru rezolvarea problemei se rezumă pe folosirea de două fișiere separate salvate ca LRS în zonele selectate de către utilizator. Pentru a asigura o consistență a conținutului acestor fișiere, de fiecare dată când un fișier este din nou uploadat, în spate se va actualiza fiecare din cele 2 fișiere salvate în Azure Cloud Storage. De asemenea, vom salva în baza de date pentru fiecare fișier timpii inițiali de update (când a început operația pentru ambele fișiere). Când

utilizatorul va dori să modifice conținutul fișierului, aplicația va stabili timpul inițial de dinainte de modificarea propriu-zisă a celor 2 fișiere în cloud. În cazul în care modificarea se realizează cu succes, pentru fiecare din cele 2 fișiere, se va modifica în baza de date ultimul timp de modificare pentru acea copie (de notat, dacă operația de modificare reușește doar pentru o singură copie atunci doar o singură intrare din baza de date va fi actualizată). Practic, se încearcă un mirroring al datelor la operația de modificare prin schimbarea în mod sincron a celor două copii de fișiere.

Poate că acest procedeu de salvat ultimul timp de update pentru fiecare copie nu pare unul prea important la prima vedere, dar este vital pentru următoarea problemă: dacă una din cele două zone de disponibilitate a suferit probleme în timpul de update (în special zona principală selectată de utilizator), atunci pentru utilizator ar fi de dorit ca să îi fie oferit conținutul fișierului cel mai recent modificat, chiar dacă acesta este cel dat de către replica din a doua zonă de redundanță selectată. Astfel, când utilizatorul face un request pentru fișier, se vor citi din baza de date cele două intrări pentru cele două copii, se va încerca aducerea conținutului pentru copia modificată cel mai recent, iar doar în cazul în care această copie nu este disponibilă, conținutul celeilalte (deși mai vechi) va fi oferit utilizatorului. Acest procedeu este vizibil în Figura 4-14.

Practic, codul prezentat realizează o cerere către fișierul cerut prin apelul `blobClient.GetPropertiesAsync()` pentru a verifica disponibilitatea acestuia, iar dacă sunt erori în cadrul acestui proces, se va încerca folosirea celui de-al doilea fișier. Pe tot parcursul implementării de interacțiuni cu fișierele stocate în Azure Storage Accounts s-a folosit Software Development Kit-ul (SDK) oferit de către Azure pentru tehnologii .NET.

De asemenea, propunem și utilizarea unei Azure Function ce va rula la un interval prestabilit de timp pentru a citi pentru fiecare fișier cele două intrări din baza de date, și a updata (dacă operația reușește cu succes) a copiilor care au ultimul timp de update mai vechi decât a celeilalte copii. Astfel, se asigură o consistență cât mai bună pentru cele două copii. Trebuie să menționăm că gradul de consistență oferit crește odată cu micșorarea intervalului de timp în care Azure Function-ul folosit, din motive evidente: dacă timpul este setat pentru un minut, atunci acest lucru înseamnă că dacă operația a eșuat, se va reîncerca modificarea pentru acea copie peste un alt minut, fiind un timp scurt pentru utilizatori în care pot acționa pe o copie invalidă (luând în considerare și șansa extrem de mică ca o copie să nu fie disponibilă iar în acel minut, cealaltă

copie să devină indisponibilă iar prima să își revină). Micșorarea timpului pentru Azure Function vine în schimb cu un cost mai ridicat, iar abordarea din fiecare minut poate fi una destul de costisitoare.

```
public async Task<AppFile> GetAvailableFileReplica(Guid fileId, bool includeVersions)
{
    AppFile originalRequestedFile = includeVersions
        ? await this._appFilesDAL.GetFullFileById(fileId)
        : await this._appFilesDAL.GetFileByIdWithStorageAccount(fileId);

    if (originalRequestedFile.ReplicaId == null)
    {
        return originalRequestedFile;
    }

    AppFile replicaFile = includeVersions
        ? await this._appFilesDAL.GetFullFileById((Guid)originalRequestedFile.ReplicaId)
        : await this._appFilesDAL.GetFileByIdWithStorageAccount((Guid)originalRequestedFile.ReplicaId);

    AppFile firstCheckedFile = originalRequestedFile.LastUpdate >= replicaFile.LastUpdate ? originalRequestedFile : replicaFile;
    AppFile secondCheckedFile = firstCheckedFile.Id == originalRequestedFile.Id ? replicaFile : originalRequestedFile;

    try
    {
        BlobServiceClient blobServiceClient = new BlobServiceClient(firstCheckedFile.StorageAccount.ConnectionString);
        BlobContainerClient containerClient = blobServiceClient.GetBlobContainerClient("container");
        BlobClient blobClient = containerClient.GetBlobClient(firstCheckedFile.Id.ToString() + Path.GetExtension(firstCheckedFile.Name));

        await blobClient.GetPropertiesAsync();
        return firstCheckedFile;
    }
    catch
    {
        try
        {
            BlobServiceClient blobServiceClient = new BlobServiceClient(secondCheckedFile.StorageAccount.ConnectionString);
            BlobContainerClient containerClient = blobServiceClient.GetBlobContainerClient("container");
            BlobClient blobClient = containerClient.GetBlobClient(secondCheckedFile.Id.ToString() + Path.GetExtension(secondCheckedFile.Name));

            await blobClient.GetPropertiesAsync();
            return secondCheckedFile;
        }
        catch
        {
            throw new Exception("File not available");
        }
    }
}
```

Figură 4-14

S-a folosit și un al doilea serviciu de tip Azure Function, care rulează săptămânal. Acesta verifică pentru fiecare fișier timestamp-ul pentru ultima interacționare cu el, iar în funcție de Access Tier-ul setat pentru fișier și de diferența dintre ultimul timp de interacțiune și timpul când rulează Function App-ul, se poate modifica Access Tier-ul. În principiu, am considerat că un fișier ar trebui să fie în Hot Tier dacă e folosit în ultimele 7 zile, iar în Cool Tier în ultimele 30 de zile, altfel să fie trecut în Archive Tier.

Așa cum am mai amintit, partea de front-end s-a realizat cu ajutorul React, în care ideea principală este de a crea componente funcționale care să fie folosite pentru interfața grafică, fiind o practică recomandată de a crea componente mai mici care să primească proprietăți diferite doar și să fie reutilizate în cadrul altor componente. S-au folosit elemente clasice precum Hook-uri și

State-uri, dar și elemente din cadrul altor librării precum Fluent UI. Hook-urile în React reprezintă o funcționalitate a acestui framework care îl face din ce în ce mai popular în rândul dezvoltatorilor. Ele permit componentelor funcționale să acceseze starea fără a fi necesară scrierea unei clase. În aplicația noastră, am folosit două hook-uri: `useState` și `useEffect`. Primul ne permite să modificăm conținutul unei pagini atunci când un obiect se schimbă (de exemplu, când un utilizator selectează dacă să folosească sau nu comparația dintre versiuni). Al doilea hook este utilizat pentru a executa o funcție ori de câte ori un obiect dintr-o listă de dependențe se modifică.

```
<Stack className={containerClassName}>
  <Modal isOpen={isModalOpen} onDismiss={() => setIsModalOpen(false)}>
    <AddFileModal
      onAddedFile={handleAddFile}
    />
  />
  </Modal>
  <Stack className={titleClassName} horizontal horizontalAlign="space-between">
    <StackItem style={{ fontSize: "25px", color: "#004e8c" }}>
      File System App
    </StackItem>
    <StackItem style={{ fontSize: "25px", color: "#004e8c" }}>
      {localStorage.getItem("userName")}
    </StackItem>
  </Stack>
  <Stack horizontal horizontalAlign="end" tokens={{ childrenGap: 20 }}>
    <button className={buttonClassName} onClick={() => setIsModalOpen(true)}>
      <Icon
        iconName="Add"
        className={iconClassName}
      />
      Add File
    </button>
    <button className={buttonClassName} onClick={handleLogout}>
      <Icon
        iconName="SignOut"
        className={iconClassName}
      />
      Logout
    </button>
  </Stack>
  {files?.length > 0 &&
    <div className={listContainerClassName}>
      <DetailsList
        items={files}
        columns={columns}
        setKey="set"
        styles={{ root: { maxHeight: "500px" } }}
        layoutMode={DetailsListLayoutMode.fixedColumns}
        selectionPreservedOnEmptyClick={true}
        selectionMode={SelectionMode.none}
      />
    </div>
  }
</Stack>
```

Figură 4-15

```
const [files, setFiles] = React.useState<IAppFile[]>([]);

const handleAddFile = (newFile: IAppFile): void => {
  setFiles([...files, newFile]);
  setIsModalOpen(false);
};
```

Figură 4-16

Un exemplu de cod pentru o componentă în React, cel puțin pe partea de construire a elementelor de afișat, este dat în Figura 4-15. Se observă utilizarea unei alte componente realizate tot în cadrul aplicației, `AddFileModal`, în care se specifică o anumită funcție de utilizat pentru când un fișier este adăugat cu succes. De asemenea, sunt folosite și componente precum

Stack ori DetailsList din cadrul Fluent UI, care permit modificări prin transmiterea de proprietăți precum cele de styling prin className. Cât despre state, files reprezintă un state cu toate fișierele utilizatorului (doar proprietăți simple, nu și conținut) care se va folosi pentru a afișa proprietățile fișierelor în cadrul unei liste. Când se adaugă cu succes un nou fișier, state-ul de files se va modifica, declanșând o re-arendare a DetailsList cu noua listă de fișiere (Figura 4-16).

FileVisualiser reprezintă un exemplu de componentă funcțională specifică React ce s-a folosit în cadrul aplicației. Aceasta a fost creată pentru a afișa diferite tipuri de fișiere, folosind numele acestuia (pentru a extrage extensia și implicit tipul fișierului), cât și token-ul SAS pentru a avea acces la conținutul fișierului (Figura 4-17).

```
export const FileVisualiser = (props: IFileVisualiserProps): JSX.Element => {
  const extension: string = props.fileName.split('.').pop()?.toLowerCase() as string;

  if (['png', 'jpg', 'jpeg', 'gif', 'bmp', 'svg', 'webp'].includes(extension)) {
    return (
      <div>
        <h3>Image Preview</h3>
        <img src={props.tokenSAS} alt={props.fileName} style={{ maxWidth: '100%', maxHeight: '80vh' }} />
      </div>
    );
  }

  if (extension === 'pdf') {
    return (
      <div>
        <h3>PDF Preview</h3>
        <iframe
          src={props.tokenSAS}
          title="PDF Viewer"
          width="100%"
          height="800px"
          style={{ border: 'none' }}
        />
      </div>
    );
  }
}
```

Figură 4-17

4.4 Testare

În ceea ce privește testarea aplicației, s-au creat mai multe scenarii pentru fiecare opțiune în parte care i se oferă utilizatorului și asigurarea unui parcurs fără erori nejustificate, ca urmare a unor probleme de cod. Astfel, s-a asigurat crearea de conturi, partea de autentificare, adăugarea de noi fișiere cu diferite opțiuni selectate privind redundanța, locația, posibilitatea de versionare,

modificarea de conținut pentru fișiere cu sau fără versionare. Pentru cele mai multe situații, procedeul este unul simplu de urmărit, fiind vorba de testare manuală, însă pe partea de redundanță custom a fost nevoie de unele ajustări.

Astfel, s-au testat două scenarii pe această parte. Prima dată, s-a creat un fișier cu acest tip de redundanță după care a fost eliminat din Storage Account-ul principal și s-a urmărit faptul că aplicația va folosi în acest caz fișierul din cel de-al doilea Storage Account (practic s-a forțat un scenariu în care fișierul principal a devenit indisponibil). Acesta scenariu a funcționat, aplicația testând, cum am mai prezentat, prima dată disponibilitatea de la cele două fișiere în funcție și de ultimul care a fost modificat, dar în acest caz ambele au fost alterate în același timp, deci fișierul principal a fost testat primul, iar fiind indisponibil a fost folosită sursa.

Al doilea caz a constituit în folosirea unui fișier tot cu redundanță custom. La momentul de modificare a conținutului, s-a modificat codul sursă pentru ca în cazul fișierului din zona principală de disponibilitate să fie aruncată o excepție. În acest mod, doar cel de-al doilea fișier a fost modificat. Aici s-a urmărit ca serviciul de Azure Function să modifice primul fișier pentru a-l aduce la forma actuală cu conținutul celui de-al doilea, situație care s-a întâmplat.

5 Concluzii

Prin această lucrare, am dorit să prezint aspecte din mai multe domenii din cadrul dezvoltării unei aplicații distribuite în cloud. Accentul a căzut în cea mai mare parte pe folosirea de servicii specifice acestor medii cloud, prezentarea arhitecturii unor astfel de soluții software, având ca scop ușurarea muncii pentru dezvoltatori, cât și prin simplitatea oferirii produsului finit către utilizatori. Scopul a fost de a răspunde unor serii de întrebări, în primă fază pe partea teoretică, iar apoi în prezentarea soluției propuse.

Primul parte a acoperit mai multe zone precum care sunt principalele caracteristici ale mediilor cloud și a aplicațiilor găzduite astfel, cum se realizează stocarea fișierelor în astfel de medii, modul în care se realizează și se poate îmbunătăți redundanța și nu în ultimul rând prezentarea unor concepte pentru crearea de interfețe intuitive pentru utilizatori neantrenați.

Cea de-a doua parte s-a concentrat pe prezentarea soluției propuse, prezentând elemente precum arhitectura aplicației, atât la nivelul serviciilor cloud folosite, cât și la nivelul software în sine, dar și elemente cheie din procesul de dezvoltare. Aplicația propusă s-a concentrat asupra rezolvării a două obiective: oferirea unei aplicații cloud pentru gestiunea fișierelor cu un design intuitiv pentru utilizatori neantrenați, dar care doresc să aibă un mai mare control asupra modului în care fișierele lor sunt stocate, cea de-a doua provocare fiind dată de oferirea posibilității utilizatorilor de a alege două zone de disponibilitate pentru stocare, opțiune ce nu se regăsește în mod natural în oferta de servicii de la mari furnizori precum Microsoft Azure sau AWS, pentru acest lucru fiind necesară crearea unui model de redundanță în spate, venit să asigure o stabilitate a datelor.

Aplicația nu este una completă, fiind posibile îmbunătățiri viitoare în ceea ce implică creșterea gradului de redundanță în cazul soluției propuse precum urmărirea fiecărei operații de update separat, dar și implementarea de noi funcționalități precum crearea unei pagini de administrator în care se poate automatiza procesul de creare de noi Azure Storage Accounts pentru noi zone sau opțiuni selectate de către utilizatori. Acest lucru ar implica de asemenea și o separare clară între modul în care cineva se autentifică în cadrul aplicației și ce drepturi i se oferă.

Bibliografie

- [1] ALAM, Tanweer. Cloud Computing and its role in the Information Technology. *IAIC Transactions on Sustainable Digital Innovation (ITSDI)*, 2021, 1.2: 108-115.
- [2] CAMPELO, R. A., et al. A brief survey on replica consistency in cloud environments. *J Internet Serv Appl* 11 [online]. 2020.
- [3] ENDO, Patricia T., et al. High availability in clouds: systematic review and research challenges. *Journal of Cloud Computing*, 2016, 5: 1-15.
- [4] HUANG, Zhen, et al. Minimizing data redundancy for high reliable cloud storage systems. *Computer Networks*, 2015, 81: 164-177.
- [5] HUI, Sarah Low Tze; SEE, Swee Lan. Enhancing user experience through customisation of UI design. *Procedia Manufacturing*, 2015, 3: 1932-1937.
- [6] ISLAM, Rafia, et al. The future of cloud computing: benefits and challenges. *International Journal of Communications, Network and System Sciences*, 2023, 16.4: 53-65.
- [7] MARCHETTO A, TONELLA P, RICCA F, “State-based testing of ajax web applications” in *1st Int. Conf. on SoftwareTesting, Verification, and Validation*, Lillehammer, Norway, pp. 121–130, 2008
- [8] MCKAY, Everett N. *UI is communication: How to design intuitive, user centered interfaces by focusing on effective communication*. Newnes, 2013.
- [9] MILANI, Bahareh Alami; NAVIMIPOUR, Nima Jafari. A comprehensive review of the data replication techniques in the cloud environments: Major trends and future directions. *Journal of Network and Computer Applications*, 2016, 64: 229-238.
- [10] PATTERSON, R. Hugo; MANLEY, Stephen. {SnapMirror}:{File-System-Based} Asynchronous Mirroring for Disaster Recovery. In: *Conference on File and Storage Technologies (FAST 02)*. 2002.

- [11] TABET, Khaoula, et al. Data replication in cloud systems: a survey. *International Journal of Information Systems and Social Change (IJISSC)*, 2017, 8.3: 17-33.
- [12] UZOMA, Benneth Chukwuemeka; OKHUOYA, Isokpehi Bonaventure. A research on cloud computing. *Research Gate*, 2022.
- [13] VADLAMUDI S G, SENGUPTA S, TAGUINOD M, et al., *Moving target defense for web applications using Bayesian stackelberg games*, in *Proc. of the 15th International Conference on Autonomous Agents and Multiagent Systems*, 1377-1378, 2016